Introduction to Deep Learning

Deep Learning is a branch of **Machine Learning (ML)** that uses **artificial neural networks with multiple layers** to learn patterns from large amounts of data.

Deep Learning is widely used in:

- Image recognition
- Speech recognition
- Natural language processing
- Recommendation systems
- Fraud detection
- Medical diagnosis
- Autonomous vehicles

Deep Learning is a machine learning technique that uses multi-layered neural networks to automatically learn complex patterns from data.

AI, Machine Learning and Deep Learning

The relationship can be represented as:

Artificial Intelligence (AI)

↓

Machine Learning (ML)

↓

Deep Learning (DL)

Artificial Intelligence

AI is the broad field of developing systems that can perform tasks that normally require human intelligence.

Machine Learning

ML allows computers to learn patterns from data and make predictions without being explicitly programmed for every task.

Deep Learning

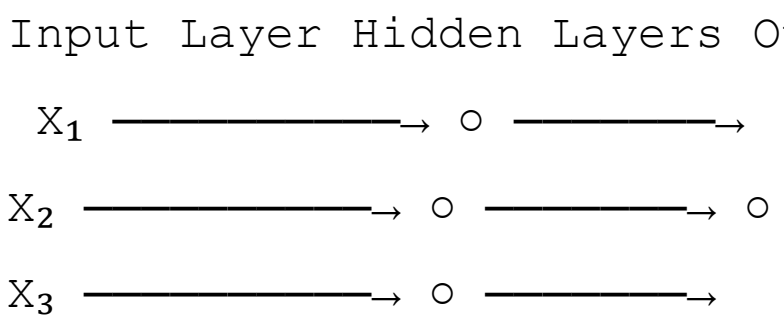
DL uses neural networks with multiple layers to learn complex patterns automatically.

What is an Artificial Neural Network?

An **Artificial Neural Network (ANN)** is a computational model inspired by the working of biological neurons.

A neural network consists mainly of:

1. Input layer
2. Hidden layer(s)
3. Output layer



Layers of a Neural Network

Input Layer

Receives the input data.

Example: For predicting house prices:

- Area
- Number of bedrooms
- Location
- Age of house

Hidden Layer

Processes the input data and identifies patterns.

A deep neural network may contain many hidden layers.

Output Layer

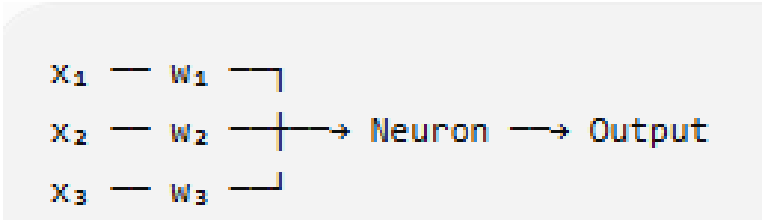
Produces the final result.

Example:

Input → Neural Network → Output Student details → Model → Pass / Fail

Artificial Neuron

An artificial neuron receives input values and processes them.



The neuron calculates a weighted sum:

$z = w_1x_1 + w_2x_2 + w_3x_3 + b$

$z = w_1x_1 + w_2x_2 + w_3x_3 + b$

where:

- **x** = input
- **w** = weight
- **b** = bias
- **z** = calculated value

An **activation function** is then applied to produce the output.

Weights and Bias

Weight

A **weight** indicates the importance of an input.

For example, in predicting whether a student will pass, attendance may have a particular weight.

Bias

A **bias** is an additional value that helps the neuron adjust its output.

Activation Function

An **activation function** determines whether and how strongly a neuron should be activated.

Common activation functions are:

ReLU

ReLU(x)=max(0,x)

Example:

Input: -5 → Output: 0

Input: 3 → Output: 3

ReLU – Rectified Linear Unit

ReLU stands for **Rectified Linear Unit**. It is one of the most commonly used **activation functions in deep learning neural networks**.

Definition

ReLU gives the input value as output if it is positive. If the input is negative, it gives **0**.

Sigmoid

Produces values between **0 and 1**.

It is commonly used for binary classification.

Example:

Cat → 0.70

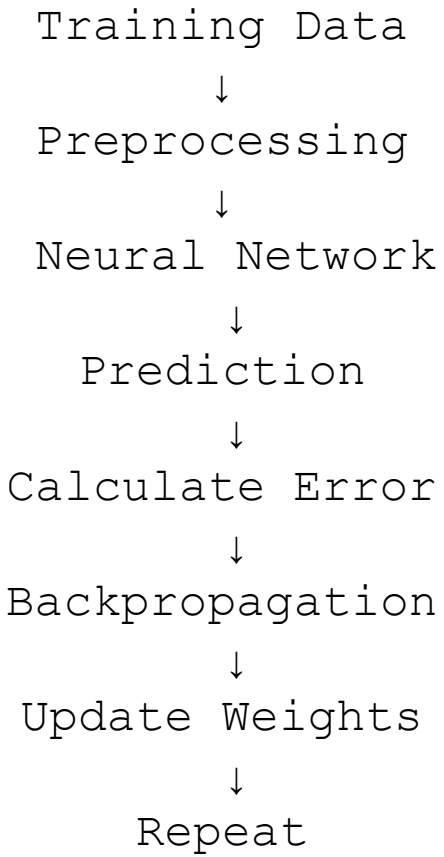
Dog → 0.20

Horse → 0.10

The model predicts **Cat**.

How Does Deep Learning Work?

The basic process is:



The model repeatedly learns from the training data and adjusts its weights to improve its predictions.

Training a Deep Learning Model

Step 1: Collect Data

Large amounts of relevant data are collected.

Step 2: Preprocess Data

Data is cleaned and converted into a suitable format.

Step 3: Train the Model

The training data is given to the neural network.

Step 4: Calculate Error

The predicted output is compared with the actual output.

Step 5: Update Weights

The model adjusts its weights using **backpropagation and an optimizer**.

Step 6: Repeat

The process is repeated until the model achieves satisfactory performance.

Loss Function

A **loss function** measures the difference between the predicted result and the actual result.

Example:

Actual result = 1 Predicted result = 0.3

There is an error between the two values.

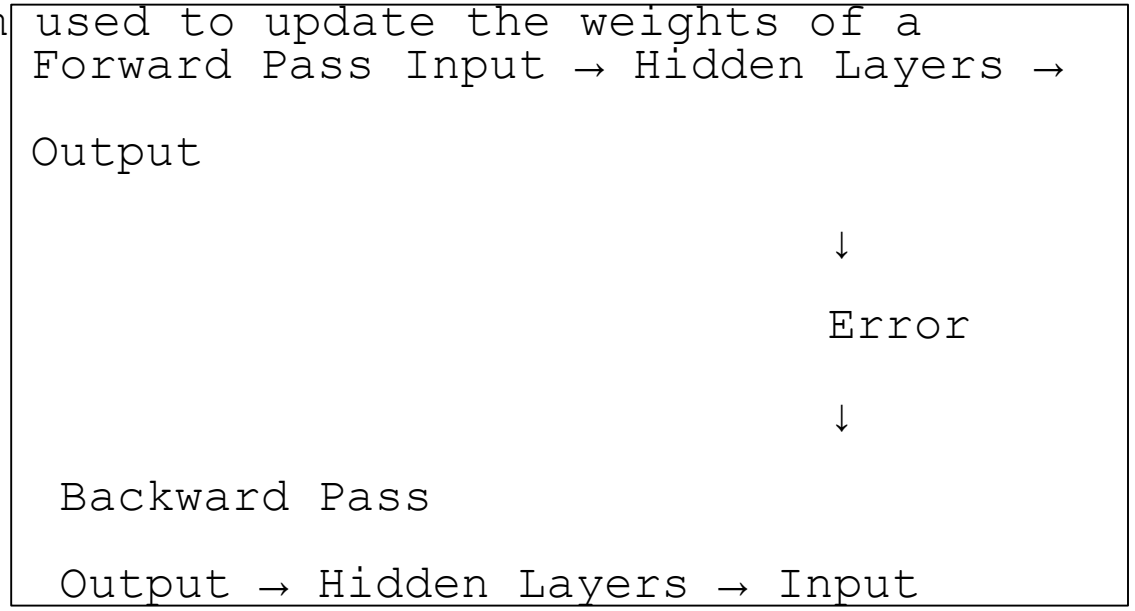
The objective of training is generally to **minimize the loss.**

Backpropagation

Backpropagation is a learning algorithm used to update the weights of a neural network.

It works by:

- Calculating the prediction.
- Calculating the error.
- Propagating the error backward.
- Adjusting the weights.



Epoch

An **epoch** means one complete pass through the entire training dataset.

For example:

If there are **1,000 training records**, processing all 1,000 records once represents **one epoch**.

Batch

A **batch** is a smaller group of training examples processed together.

Example:

Training data = 1,000 records

Batch size = 100

100 records → Model

100 records → Model

...

Common Deep Learning Models

CNN – Convolutional Neural Network

CNN is mainly used for **image and visual data.**

Applications:

- Face recognition
- Object detection
- Medical image analysis
- Handwriting recognition

Image → Convolution → Pooling → Neural Network → Result

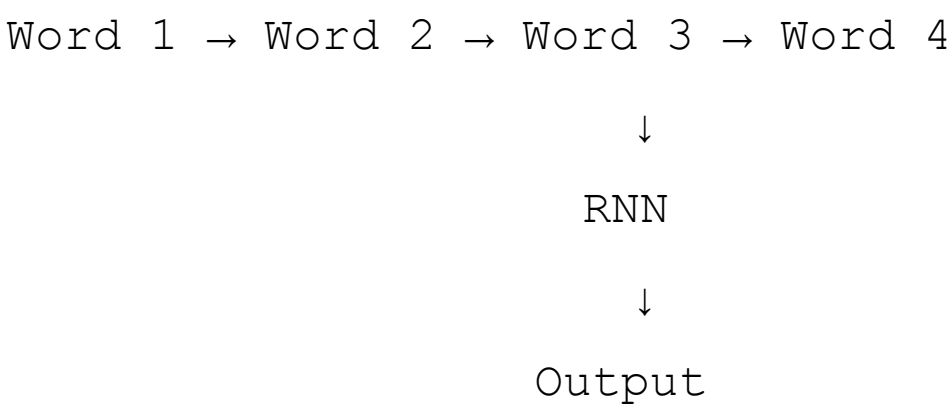
RNN – Recurrent Neural Network

RNN is designed for **sequential data.**

Applications:

- Text processing
- Speech processing
- Time-series prediction

Example:



LSTM

Long Short-Term Memory (LSTM) is a type of RNN that can remember important information for longer periods.

Applications:

- Speech recognition
- Language processing
- Time-series prediction

Transformer

A **Transformer** is a modern deep-learning architecture that uses an **attention mechanism** to understand relationships between elements in a sequence.

It is widely used in:

- Natural language processing
- Machine translation
- Text generation
- Code analysis

Advantages of Deep Learning

- Can learn complex patterns.
- Can process large amounts of data.
- Reduces the need for manual feature extraction in many applications.
- Can achieve high accuracy for suitable tasks.
- Useful for images, text, speech and other complex data.
- Can automatically learn representations from raw or minimally processed data.

Limitations of Deep Learning

- Requires large amounts of data for many applications.
- Training can require significant computational resources.
- Training can take considerable time.
- Models can be difficult to interpret.
- Poor-quality training data can produce poor results.
- Deep-learning models can suffer from overfitting.

Building and Training Models

Building and training a machine learning model means creating a model that can learn patterns from data and then use those patterns to make predictions or decisions.

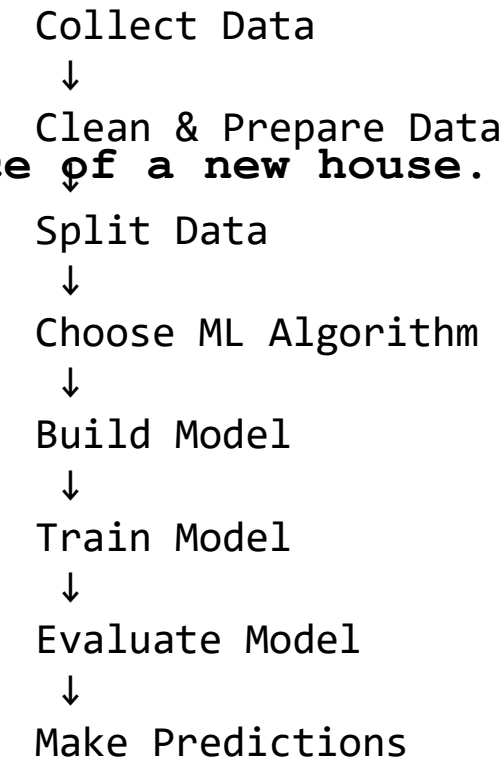
1. What is a Machine Learning Model?

A **machine learning model** is a program that learns patterns from existing data and uses those patterns to predict outcomes for new data.

Example:

A model can learn from previous house prices and predict the price of a new house.

Basic process:



Building a Model

Building a model involves selecting a suitable machine learning algorithm and defining how the model will learn from the data.

Common algorithms include:

Task	Algorithms
Classification	Logistic Regression, Decision Tree, KNN
Regression	Linear Regression, Decision Tree
Clustering	K-Means
Text Classification	Naive Bayes
Complex problems	Neural Networks

Training a Model

Training is the process where the machine learning algorithm learns from the training data.

For example, suppose we want to predict whether a student will pass:

Study Hours → Attendance→ Previous Marks

↓

Training Data

↓

Machine Learning Algorithm

↓

Trained Model

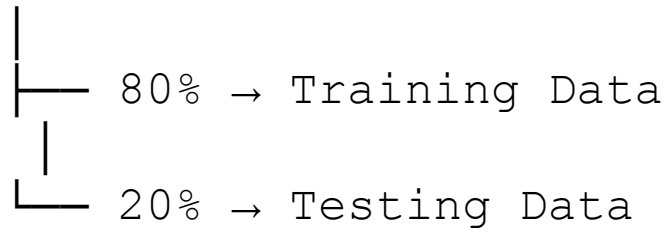
The model identifies relationships between the input features and the expected output.

Training and Testing Data

Usually, the dataset is divided into two parts:

- **Training data** – used to teach the model.
- **Testing data** – used to check how well the trained model performs on unseen data.

A common split is:



- X → input/features
- y → target/output
- LinearRegression() → creates the model
- fit() → trains the model
- predict() → makes predictions

Features: Input variables used by the model.

Target: The output that the model tries to predict.

Training: Learning patterns from training data.

Testing: Evaluating the model using unseen data.

Python

```python
from sklearn.linear_model import LinearRegression

# Training data
X = [[1], [2], [3], [4], [5]]
y = [2, 4, 6, 8, 10]

# Create the model
model = LinearRegression()

# Train the model
model.fit(X, y)

# Make a prediction
prediction = model.predict([[6]])

print(prediction)
```